

Punto 2.2 - Entidades y propiedades

Ejercicio guiado: enriquecer las entidades de `AceriaData` y configurar explícitamente sus propiedades escalares: requeridos, opcionales, longitudes máximas, precisión decimal y valores por defecto.

Contexto del proyecto: en 2.1 se inspeccionó el modelo que EF Core había construido por convención. Ahora se parte de ese mismo estado y se modifica el modelo de forma controlada. Este punto se limita a **entidades y propiedades**. Las relaciones explícitas se trabajarán desde 2.3, las relaciones muchos-a-muchos en 2.5 y los índices y restricciones de negocio en 2.9.

Objetivos de aprendizaje

- Añadir propiedades de negocio sin reiniciar `AceriaData`.
- Distinguir propiedades requeridas y opcionales.
- Configurar longitudes máximas para cadenas.
- Configurar precisión y escala para valores `decimal`.
- Configurar valores por defecto constantes y generados por SQL Server.
- Comprender qué cambios de propiedades producen un delta de esquema.
- Generar y revisar una migración incremental sobre el snapshot heredado.
- Verificar que el punto no introduce relaciones ni índices reservados para puntos posteriores.

Paso 1: Abrir el estado autónomo de 2.2

```
cd M02/PROYECTO/2.2
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

El checkpoint 2.2 ya contiene el resultado completo del ejercicio. Si estás siguiendo el laboratorio manualmente desde 2.1, aplica sobre tu copia los cambios descritos en los pasos siguientes y genera después la migración incremental.

Paso 2: Añadir Peso y Activa a `PlanchaAcero`

La entidad incorpora dos propiedades de negocio nuevas:

```
public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

- `Peso` usa `decimal` porque en este dominio interesa conservar una escala decimal controlada.
- `Activa` representa el estado operativo de la plancha y parte de `true`.
- `OrdenId` y la navegación `Orden` ya existían; en este punto no se reconfigura todavía la relación.

El error que debe evitarse aquí es aprovechar el cambio para configurar `HasOne/WithMany`: esa configuración pertenece a 2.3.

Paso 3: Añadir propiedades de negocio a `OrdenFabricacion`

El estado 2.2 amplía la orden con fecha de entrega, estado y observaciones:

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}
```

FechaEntrega es DateTime? porque una orden puede crearse antes de disponer de una fecha definitiva. Observaciones es string? por la misma razón: no toda orden necesita texto adicional.

Si FechaEntrega se declarase como DateTime, el modelo la trataría como no anulable y se perdería esa regla del dominio.

| Paso 4: Completar Aleacion y EstadoOrden sin adelantar relaciones

También se incorporan propiedades escalares a las otras entidades:

```
public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}
```

La fuente original proponía en este punto colecciones de navegación para una relación muchos-a-muchos. Esa relación **no se introduce en 2.2**: se reserva para 2.5, donde se modelará con la entidad intermedia OrdenAleacion.

| Paso 5: Configurar las propiedades de OrdenFabricacion

En OnModelCreating, el checkpoint configura únicamente aspectos que pertenecen a propiedades:

```

modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);

    entity.Property(o => o.NumeroOrden)
        .IsRequired()
        .HasMaxLength(50);

    entity.Property(o => o.Cliente)
        .IsRequired()
        .HasMaxLength(200);

    entity.Property(o => o.FechaCreacion)
        .HasDefaultValueSql("GETDATE()");

    entity.Property(o => o.Estado)
        .IsRequired()
        .HasMaxLength(50)
        .HasDefaultValue("Pendiente");

    entity.Property(o => o.Observaciones)
        .HasMaxLength(500);
});

```

Qué debes observar:

1. `NumeroOrden` y `Cliente` dejan de ser cadenas sin límite conocido.
2. `FechaCreacion` obtiene un valor por defecto generado por SQL Server.
3. `Estado` es requerido, tiene longitud máxima y valor por defecto.
4. `Observaciones` conserva la nulabilidad del tipo CLR y limita su longitud.
5. No se crea todavía un índice único para `NumeroOrden`; los índices se estudian en 2.9.

| Paso 6: Configurar precisión y valor por defecto de PlanchaAcero

La configuración relevante de la plancha es:

```

modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable("PlanchasAcero");
    entity.HasKey(x => x.Id);

    entity.Property(x => x.Peso)
        .HasPrecision(18, 3);

    entity.Property(x => x.Activa)
        .HasDefaultValue(true);
});

```

`HasPrecision(18, 3)` significa hasta 18 dígitos en total, de los cuales 3 quedan a la derecha del separador decimal. El objetivo es que valores como `371.250` se representen con la escala prevista.

`HasDefaultValue(true)` define el valor por defecto a nivel de modelo/base de datos. Si una operación envía explícitamente otro valor, prevalece el valor enviado.

| Paso 7: Configurar las propiedades escalares de Aleacion y EstadoOrden

El checkpoint mantiene la configuración de estas entidades centrada en propiedades:

```

modelBuilder.Entity<Aleacion>(entity =>
{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
});

modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});

```

En este estado `Codigo` tiene longitud máxima, pero **todavía no tiene un índice único**. La unicidad e índices de negocio se formalizarán en 2.9.

Paso 8: Generar la migración incremental desde 2.1

Si estás reproduciendo el ejercicio manualmente sobre una copia del estado 2.1, después de introducir los cambios anteriores ejecuta:

```
dotnet ef migrations add M2_2_2
```

`migrations add` compara el modelo modificado con el snapshot heredado y genera el delta necesario.

En el checkpoint entregado **no debes volver a ejecutar ese comando**, porque la migración ya existe:

```
Migrations/20260927204717_M2_2_2.cs
```

Generarla de nuevo sobre el estado final produciría otra migración distinta y dejaría de representar el proceso incremental original.

Paso 9: Revisar qué cambia realmente la migración M2_2_2

Abre `Migrations/20260927204717_M2_2_2.cs` antes de aplicar nada. Entre sus operaciones reales están:

- añadir `Activa` y `Peso` a `PlanchasAcero`;
- cambiar `NumeroOrden` a `nvarchar(50)`;
- cambiar `Cliente` a `nvarchar(200)`;
- establecer `GETDATE()` como valor por defecto de `FechaCreacion`;
- añadir `Estado`, `FechaEntrega` y `Observaciones`;
- añadir `Activo` a `EstadosOrden`;
- añadir `Codigo` y `Descripcion` a `Aleaciones`;
- aplicar las longitudes máximas configuradas.

Por ejemplo, la columna `Peso` aparece como:

```

migrationBuilder.AddColumn<decimal>(
    name: "Peso",
    table: "PlanchasAcero",
    type: "decimal(18,3)",
    precision: 18,
    scale: 3,
    nullable: false,
    defaultValue: 0m);

```

La migración de 2.2 **no debe contener índices de negocio ni la configuración explícita de nuevas relaciones**. Si aparecen, se ha adelantado contenido posterior.

Paso 10: Aplicar la migración y verificar el esquema

Sobre el laboratorio construido desde el estado anterior:

```
dotnet ef database update
dotnet ef migrations list
```

Comprueba en SQL Server Object Explorer, al menos:

- `OrdenesFabricacion.NumeroOrden` → longitud máxima 50;
- `OrdenesFabricacion.Cliente` → longitud máxima 200;
- `OrdenesFabricacion.FechaEntrega` → admite `NULL`;
- `OrdenesFabricacion.Observaciones` → admite `NULL`;
- `PlanchasAcero.Peso` → `decimal(18,3)`;
- `PlanchasAcero.Activa` → `bit` con valor por defecto;
- las nuevas propiedades de `Aleaciones` y `EstadosOrden`.

El flujo del curso está gobernado por **Migrations**. No se usa `EnsureCreated()` para sustituir este historial.

Paso 11: Ejecutar el estado completo y comprobar sus datos

Ejecuta:

```
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

El programa aplica el historial con `Database.Migrate()`, inserta una orden de demostración y termina mostrando una evidencia equivalente a:

2.2 OK | Órdenes: 1 | Planchas: 1

La plancha de prueba utiliza `Peso = 371.250m`, de forma que la ejecución atraviesa realmente la propiedad cuya precisión se ha configurado.

Errores comunes

Error	Causa	Corrección
La migración sale vacía	El modelo no cambió respecto al snapshot	Revisar que las propiedades y su configuración se hayan incorporado antes de generar la migración
<code>FechaEntrega</code> no admite <code>NULL</code>	Se declaró como <code>DateTime</code>	Usar <code>DateTime?</code>
<code>Peso</code> no conserva la escala prevista	Falta <code>HasPrecision(18, 3)</code>	Configurar precisión y escala antes de generar la migración
Aparecen índices únicos en 2.2	Se adelantó contenido de 2.9	Mantener en 2.2 sólo configuración de propiedades
Aparece una nueva relación muchos-a-muchos	Se recuperó literalmente un ejemplo antiguo	Reservar <code>OrdenAleacion</code> para 2.5
Se usa <code>EnsureCreated()</code>		

Error	Causa	Corrección
	Se sustituye el historial de migraciones por creación directa	Usar la migración incremental y Database.Migrate() / database update

Retos resueltos: comprobar el modelo mediante metadatos

Retos: verificar desde EF Core que `Observaciones` sigue siendo opcional y que `Peso` tiene precisión 18 y escala 3.

El checkpoint contiene un bloque pedagógico comentado `RETO 2.2 - COMPROBAR PROPIEDADES`. Al activarlo se consultan los metadatos del modelo:

```
var ordenType = context.Model.FindEntityType(typeof(OrdenFabricacion))
?? throw new InvalidOperationException("No se encontró OrdenFabricacion en el modelo.");
var observaciones = ordenType.FindProperty(nameof(OrdenFabricacion.Observaciones))
?? throw new InvalidOperationException("No se encontró Observaciones en el modelo.");

var planchaType = context.Model.FindEntityType(typeof(PlanchaAcero))
?? throw new InvalidOperationException("No se encontró PlanchaAcero en el modelo.");
var peso = planchaType.FindProperty(nameof(PlanchaAcero.Peso))
?? throw new InvalidOperationException("No se encontró Peso en el modelo.");

global::System.Console.WriteLine($"Observaciones nullable: {observaciones.IsNullable}");
global::System.Console.WriteLine(
    $"Peso precision/scale: {peso.GetPrecision()}/{peso.GetScale()}");
```

El resultado esperado debe confirmar:

```
Observaciones nullable: True
Peso precision/scale: 18/3
```

Este reto no añade índices, relaciones ni propiedades de puntos posteriores.

Analogía final

Configurar propiedades se parece a fijar las especificaciones dimensionales de una pieza antes de fabricarla. No basta con saber que existe un campo “peso”: hay que decidir con qué precisión se registra. No basta con tener un “número de orden”: conviene limitar su longitud. Y una fecha de entrega puede no conocerse todavía, del mismo modo que una orden de producción puede abrirse antes de cerrar todos sus datos logísticos. El modelo convierte esas decisiones del dominio en reglas persistentes.

Resultado esperado

Al finalizar 2.2, `AceriaData` conserva el estado heredado de 2.1 y añade únicamente el delta de entidades y propiedades. La migración `M2_2_2` representa ese cambio, el esquema se aplica mediante Migrations, el programa termina con `2.2 OK` y el reto confirma la nulabilidad de `Observaciones` y la precisión 18/3 de `Peso`.

Conexión con el siguiente punto

Con las propiedades ya definidas y persistidas, 2.3 podrá centrarse exclusivamente en hacer explícita la relación uno-a-muchos entre `OrdenFabricacion` y `PlanchaAcero`. No se vuelve a crear el proyecto: se continúa desde este estado.

Código acumulativo completo del estado 2.2

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        });
    }
}

```

```

        entity.Property(o => o.Observaciones).HasMaxLength(500);
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
}

}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
    }
}

```

```

        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] {
                    "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
        };

        context.OrdenesFabricacion.Add(orden);
        context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
        Activo = true });
        context.SaveChanges();

        global::System.Console.WriteLine($"2.2 OK | Órdenes: {context.OrdenesFabricacion.Count()} |
        Planchas: {context.PlanchasAcero.Count()}");
    }
}

```

```
}
}
```

Explicación línea a línea del código acumulativo 2.2

Las referencias siguientes usan la numeración real de `M02/PROYECTO/2.2/Program.cs`. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `public class OrdenFabricacion` → declara la clase `OrdenFabricacion` que forma parte del estado acumulativo del ejercicio.

Línea 11: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 12: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 13: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 14: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad `Cliente` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `public DateTime FechaCreacion { get; set; }` → declara la propiedad `FechaCreacion` de tipo `DateTime` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 16: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad `FechaEntrega` de tipo `DateTime?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 17: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad `Estado` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 18: `public string? Observaciones { get; set; }` → declara la propiedad `Observaciones` de tipo `string?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 19: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad `Planchas` de tipo `List` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 20: `}` → cierra el bloque de código actual.

Línea 22: `public class PlanchaAcero` → declara la clase `PlanchaAcero` que forma parte del estado acumulativo del ejercicio.

Línea 23: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 24: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 25: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `public double Espesor { get; set; }` → declara la propiedad `Espesor` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 27: `public double Ancho { get; set; }` → declara la propiedad `Ancho` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 28: `public double Largo { get; set; }` → declara la propiedad `Largo` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 29: `public decimal Peso { get; set; }` → declara la propiedad `Peso` de tipo `decimal` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 30: `public bool Activa { get; set; } = true;` → declara la propiedad `Activa` de tipo `bool` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 31: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `}` → cierra el bloque de código actual.

Línea 34: `public class Aleacion` → declara la clase `Aleacion` que forma parte del estado acumulativo del ejercicio.

Línea 35: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 36: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 37: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad `Nombre` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 38: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad `Codigo` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 39: `public double PorcentajeCarbono { get; set; }` → declara la propiedad `PorcentajeCarbono` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public double PorcentajeManganeso { get; set; }` → declara la propiedad `PorcentajeManganeso` de tipo `double` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public string? Descripcion { get; set; }` → declara la propiedad `Descripcion` de tipo `string?` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `}` → cierra el bloque de código actual.

Línea 44: `public class EstadoOrden` → declara la clase `EstadoOrden` que forma parte del estado acumulativo del ejercicio.

Línea 45: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 46: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 47: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad `Nombre` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 48: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad `Descripcion` de tipo `string` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 49: `public bool Activo { get; set; } = true;` → declara la propiedad `Activo` de tipo `bool` que EF Core o la lógica de negocio utiliza en este modelo.

Línea 50: `}` → cierra el bloque de código actual.

Línea 52: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo del ejercicio.

Línea 53: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 54: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método `AceriaDbContext` con los parámetros necesarios para este componente.

Línea 56: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 57: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 58: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 59: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 62: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 63: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 64: `base.OnModelCreating(modelBuilder);` → sobrescribe `OnModelCreating` para definir configuración explícita del modelo.

Línea 65: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 66: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 67: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 68: `entity.HasKey(o => o.Id);` → define la clave primaria de la entidad.

Línea 69: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 70: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 71: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 72: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 73: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 74: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 75: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 76: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 77: `entity.ToTable("PlanchasAcero");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 78: `entity.HasKey(x => x.Id);` → define la clave primaria de la entidad.

Línea 79: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 80: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 81: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 82: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 83: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 84: `entity.ToTable("Aleaciones");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 85: `entity.HasKey(a => a.Id);` → define la clave primaria de la entidad.

Línea 86: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 87: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 88: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 89: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 90: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad del modelo para aplicar su configuración Fluent API.

Línea 91: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 92: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla de SQL Server usada por la entidad.

Línea 93: `entity.HasKey(e => e.Id);` → define la clave primaria de la entidad.

Línea 94: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 95: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 96: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 97: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 100: `}` → cierra el bloque de código actual.

Línea 101: `}` → cierra el bloque de código actual.

Línea 103: `public interface IOrdenRepositorio` → declara el contrato IOrdenRepositorio para desacoplar consumidores e implementación.

Línea 104: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 105: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 106: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 107: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio NumeroOrden.

Línea 108: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 109: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 110: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 111: `}` → cierra el bloque de código actual.

Línea 113: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo del ejercicio.

Línea 114: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 115: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 116: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método `OrdenRepositorio` con los parámetros necesarios para este componente.

Línea 118: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 119: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 120: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o `null` cuando no existe.

Línea 121: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 122: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 123: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 124: `}` → cierra el bloque de código actual.

Línea 126: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` para desacoplar consumidores e implementación.

Línea 127: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 128: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 129: `}` → cierra el bloque de código actual.

Línea 131: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 132: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 133: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 134: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 135: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método ServicioOrdenes con los parámetros necesarios para este componente.

Línea 136: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 137: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 138: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 139: `}` → cierra el bloque de código actual.

Línea 141: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 142: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 143: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 144: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 145: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 146: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 147: `}` → cierra el bloque de código actual.

Línea 148: `}` → cierra el bloque de código actual.

Línea 150: `public static class Program` → declara la clase Program que forma parte del estado acumulativo del ejercicio.

Línea 151: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 152: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 153: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 154: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 155: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 156: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json a la configuración.

Línea 157: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 158: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 160: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión AceriaDB desde la configuración.

Línea 161: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 163: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 164: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en el contenedor de inyección de dependencias.

Línea 165: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de `AddDbContext`.

Línea 166: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 167: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 168: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor SQL Server.

Línea 169: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos SQL.

Línea 170: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 171: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de EF Core.

Línea 172: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 174: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 175: `services.AddScoped<IServicio0rdenes, Servicio0rdenes>();` → registra un servicio scoped para compartir dependencias dentro del mismo ámbito.

Línea 177: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante `using var`.

Línea 178: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 179: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 180: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 181: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 183: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 184: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 185: `context.Database.EnsureDeleted();` → elimina la base del laboratorio para reproducir el escenario desde un estado limpio.

Línea 186: `context.Database.Migrate();` → aplica las migraciones pendientes y deja AceriaDB en el esquema del punto.

Línea 188: `var orden = new OrdenFabricacion` → crea la entidad OrdenFabricacion que concentra el grafo de datos del escenario acumulativo.

Línea 189: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 190: `NumeroOrden = "0F-M2-0001"`, → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 191: `Cliente = "Constructora del Norte"`, → asigna el cliente de la orden utilizada en el escenario E2E del punto.

Línea 192: `FechaCreacion = DateTime.UtcNow`, → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 193: `FechaEntrega = DateTime.Today.AddDays(14)`, → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 194: `Estado = "Pendiente"`, → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 195: `Observaciones = "Orden de validación M2"`, → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 196: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 197: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 198: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 199: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 200: `Espesor = 10.5`, → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 201: `Ancho = 1500`, → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 202: `Largo = 3000`, → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 203: `Peso = 371.250m`, → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 204: `Activa = true`, → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 205: `}` → cierra el bloque de código actual.

Línea 206: `},` → continúa el inicializador o la llamada iniciada en el bloque actual.

Línea 207: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 209: `context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 210: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true });` → registra una entidad o relación nueva en el ChangeTracker para insertarla al guardar.

Línea 211: `context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el DbContext.

Línea 213: `global::System.Console.WriteLine($"2.2 OK | Órdenes: {context.OrdenesFabricacion.Count()} | Planchas: {context.PlanchasAcero.Count()}");` → imprime el marcador E2E de 2.2 junto con los recuentos persistidos de órdenes y planchas.

Línea 214: `}` → cierra el bloque de código actual.

Línea 215: `}` → cierra el bloque de código actual.